

UNIFECAP • GAME DEVELOPMENT

# LUMEN

Relatório Teórico - Plataforma 2D Cyberpunk na Unity

**Aluno:** Samuel Nunes

**Disciplina:** Game Development

**Engine:** Unity 6 (6000.4) • **Linguagem:** C#

**Data:** 14 de junho de 2026

# 1. Introdução e proposta do jogo

---

Neste relatório eu apresento o **Lumen**, o jogo de plataforma 2D que desenvolvi para a disciplina, com uma estética **cyberpunk** (neon synthwave). A protagonista é a **Luna**, que precisa atravessar os níveis de uma megacidade futurista: saltando por abismos, escalando, coletando *data-shards*, pegando *armas* para atirar em drones e bots de segurança, até enfrentar o **MAINFRAME** (o chefe final) numa arena. A cada fase, o céu da cidade muda de cor e pisca em neon, o que ajuda a passar a ideia de progressão.

Aqui eu explico os fundamentos teóricos que apliquei no jogo e ligo cada um deles às decisões que tomei na prática. Fiz tudo na **Unity 6** com **C#**, com a ideia de usar o projeto como peça de portfólio, mostrando o que aprendi de engine, lógica das mecânicas, design de níveis e experiência do usuário.

## Referência de mercado

Como exemplo concreto de solução já consolidada no mercado, tomamos **Celeste** (Maddy Makes Games, 2018), um dos platformers 2D mais aclamados da última década. Celeste é referência justamente porque eleva ao máximo aquilo que estudamos: *movimento responsivo*, *mecânica de escalada*, *dificuldade crescente cuidadosamente dosada* e *feedback audiovisual imediato*. Vários recursos de "game feel" usados em Celeste - como o *coyote time* e o *pulo de altura variável* - foram reproduzidos em Lumen, conforme detalhado na Seção 6. Outras referências clássicas do gênero incluem *Super Mario Bros.* (Nintendo, 1985), que fundou a gramática do platformer, e *Hollow Knight* (Team Cherry, 2017), referência de atmosfera e exploração vertical.

# 2. Princípios de jogos de plataforma 2D

---

No fundo, um jogo de plataforma 2D é um desafio de movimento e física: o personagem anda por um cenário cheio de superfícies sólidas (as plataformas), separadas por vãos, e a graça vem da mistura de *gravidade*, *impulso de pulo* e *tempo de reação*. Os principais princípios do gênero que estudei foram:

- **Gravidade e colisão:** o personagem é constantemente puxado para baixo e só para ao tocar uma superfície sólida. A detecção de chão precisa ser confiável.
- **Controle responsivo ("game feel"):** a sensação de controle é mais importante que o realismo físico. Pequenas tolerâncias (coyote time, buffer de pulo) fazem o jogo parecer "justo".

- **Leitura visual clara:** o jogador precisa bater o olho e já saber o que é chão, perigo, coletável e inimigo. No Lumen eu resolvi isso com cores neon bem diferentes entre si (faixa ciano no topo do chão, data-shards cianos brilhando, espinhos magenta e inimigos escuros com olhos vermelhos).
- **Risco e recompensa:** caminhos perigosos costumam guardar mais coletáveis, incentivando o domínio das mecânicas.
- **Curva de aprendizado:** cada habilidade é ensinada num contexto seguro antes de ser cobrada em situações difíceis.

### 3. Planejamento do projeto e mecânicas propostas

Comecei o planejamento listando os requisitos obrigatórios (andar, correr, pular, escalar; de 3 a 5 fases; HUD; feedback; arte; áudio) e tendo em mente o tempo curto que eu tinha. A decisão mais importante que tomei foi montar tudo de forma **dirigida por código**: em vez de ficar arrastando objeto por objeto no editor, o jogo inteiro é construído na hora a partir de dados (os mapas de fase em texto) e de arte/áudio gerados por código. Isso quase zerou o trabalho manual no editor e fez o projeto rodar em qualquer máquina sem depender de arquivos externos.

#### Mecânicas implementadas

| Mecânica | Descrição                                                     | Tecla                  |
|----------|---------------------------------------------------------------|------------------------|
| Andar    | Movimento horizontal em velocidade base.                      | ← → ou A / D           |
| Correr   | Aumenta a velocidade, exigido para vencer vãos maiores.       | Shift (segurar)        |
| Pular    | Pulo com altura variável e gravidade ajustada na queda.       | Espaço / W             |
| Escalar  | Sobe/desce cipós, desativando a gravidade enquanto agarrado.  | ↑ ↓ ou W / S           |
| Atirar   | Após pegar uma arma, dispara projéteis neon na direção atual. | J ou clique            |
| Pisão    | Derrota inimigos ao cair sobre eles, com pequeno repique.     | (cair sobre o inimigo) |

Há ainda dois tipos de inimigo (bots terrestres e *drones* voadores), itens de **arma** que concedem munição, e um **chefe final** com barra de vida, movimento e ataques à distância, exigindo o domínio combinado de movimento e tiro.

## 4. Abordagem de design de níveis com progressão de dificuldade

---

Foram criadas **cinco fases** com dificuldade crescente, cada uma introduzindo ou intensificando um desafio. Os níveis são descritos como **mapas ASCII** (uma matriz de caracteres), e um construtor ( `LevelBuilder` ) traduz cada símbolo em objetos do jogo. Essa técnica torna o design de fases rápido, legível e fácil de balancear.

| Fase            | Tema          | Novidade / desafio                                                     |
|-----------------|---------------|------------------------------------------------------------------------|
| 1 - Boot        | Tutorial      | Terreno plano; ensina andar/pular/coletar; primeira arma e drone.      |
| 2 - Submundo    | Abismos       | Fossos que exigem pulo; primeiros espinhos; escalada obrigatória.      |
| 3 - Arranha-Céu | Verticalidade | Mais fossos, drones e uma torre de escalada longa.                     |
| 4 - O Núcleo    | Síntese       | Combina tudo: muitos fossos, espinhos junto a saltos, vários inimigos. |
| 5 - MAINFRAME   | Chefe         | Arena do chefe final: pegar armas, desviar de tiros e destruir o boss. |

A progressão segue o princípio de "*ensinar, depois testar*": a Fase 1 apresenta cada elemento num ambiente sem punição; as fases seguintes recombina esses elementos em densidade crescente. Os vãos foram limitados a 3 blocos de largura (sempre transponíveis com um pulo) e as plataformas posicionadas em alturas comprovadamente alcançáveis, garantindo que toda fase seja **solucionável**. Cristais em arco sobre os abismos recompensam o jogador que pula com precisão (risco e recompensa).

## 5. Conceitos de interatividade, HUD e feedback

---

**Interatividade** é o ciclo entre a ação do jogador e a resposta do jogo. Quanto menor a latência e mais clara a resposta, melhor a experiência. Lumen oferece resposta imediata em três canais simultâneos: movimento do personagem, efeito visual e efeito sonoro.

### HUD (Heads-Up Display)

O HUD foi construído por código com o sistema de UI da Unity (Canvas + Text) e exibe as três informações essenciais exigidas: **vidas** (ícone de coração), **cristais coletados** na fase (ícone de

cristal, formato "atual / total") e **pontuação**. Há ainda um *banner* com o nome da fase no início de cada nível e mensagens centrais para "Fase Concluída", "Game Over" e "Você Venceu!".

## Feedback

- **Visual:** partículas douradas ao coletar cristal; piscar de invulnerabilidade ao tomar dano; inimigo "achatado" ao ser pisado; portal de saída que pulsa.
- **Sonoro:** som distinto para pulo, coleta, dano, pisão, conclusão de fase e vitória (ver Seção 7).
- **Sistêmico:** ao perder uma vida em queda, o jogador renasce no início da fase; ao zerar as vidas, ocorre Game Over com opção de recomeçar (tecla R).

## 6. Lógica de programação em C# e estrutura do game loop

O código está organizado em cerca de **20 scripts C#** com responsabilidades bem definidas (princípio da responsabilidade única). Os principais são:

| Script                                                    | Responsabilidade                                                    |
|-----------------------------------------------------------|---------------------------------------------------------------------|
| Bootstrap                                                 | Ponto de entrada; inicia o jogo automaticamente ao carregar a cena. |
| GameManager                                               | Singleton; controla vidas, pontuação, fases, vitória/derrota.       |
| PlayerController                                          | Mecânicas de movimento e animação do personagem.                    |
| LevelData / LevelBuilder                                  | Dados das fases (ASCII) e sua construção em objetos.                |
| Enemy , Boss , Projectile , WeaponPickup                  | Inimigos/drones, chefe, tiros e armas.                              |
| Collectible , Hazard , LevelExit , Ladder , SkyController | Coletáveis, perigos, saída, escada e flicker do céu.                |
| HUDController                                             | Interface do usuário.                                               |
| AudioManager                                              | Síntese de áudio (trilha e efeitos).                                |
| SpriteFactory / SpriteAnimator                            | Geração e animação de sprites.                                      |
| CameraFollow                                              | Câmera que segue o jogador dentro dos limites da fase.              |

## O game loop na Unity

A Unity executa um *game loop* em que, a cada quadro, são chamados métodos de ciclo de vida dos componentes. O projeto separa corretamente as responsabilidades entre eles:

- `Update()` - leitura de entrada e lógica por quadro (a cada frame).
- `FixedUpdate()` - física e movimento (passo de tempo fixo), evitando que a jogabilidade dependa da taxa de quadros.
- `LateUpdate()` - movimento da câmera, executado após tudo se mover.

Para alcançar um controle "justo", o `PlayerController` implementa técnicas profissionais de *game feel*:

```
// Coyote time: ainda é possível pular por um breve instante após sair da borda
if (_grounded) _coyoteTimer = coyoteTime;
else if (_coyoteTimer > 0f) _coyoteTimer -= Time.fixedDeltaTime;

// Jump buffer: registra o pulo um instante antes de tocar o chão
if (_bufferTimer > 0f && _coyoteTimer > 0f) DoJump();

// Gravidade aumentada na queda e pulo curto ao soltar o botão
if (_rb.linearVelocity.y < 0f)
    _rb.linearVelocity += Vector2.up * Physics2D.gravity.y * (fallMultiplier - 1f) *
```

A detecção de chão usa `Physics2D.OverlapBox` sob os pés do personagem, e a identificação dos elementos (cristal, inimigo, perigo) é feita por *tipo de componente* em vez de *tags*, o que torna o sistema mais robusto. O padrão **Singleton** no `GameManager` e no `AudioManager` dá acesso global e centralizado ao estado do jogo e ao áudio.

## 7. Estratégias de inserção de áudio e efeitos sonoros

Todo o áudio de Lumen é **sintetizado por código** em tempo de execução, sem arquivos externos. O `AudioManager` gera formas de onda (principalmente onda quadrada, estilo "chiptune") e as converte em `AudioClip` via `AudioClip.Create` + `SetData`. Cada evento de jogo dispara um som característico:

| Evento            | Som                                        |
|-------------------|--------------------------------------------|
| Pulo              | Varredura de frequência ascendente, curta. |
| Coleta de cristal | Dois tons agudos ("bling").                |
| Dano              | Varredura descendente com ruído.           |

|                                 |                                            |
|---------------------------------|--------------------------------------------|
| Pisão em inimigo                | Tom grave curto.                           |
| Tiro / pegar arma               | Varredura aguda rápida / arpejo de coleta. |
| Dano no chefe / chefe destruído | Ruído grave / sequência descendente longa. |
| Fase concluída / Vitória        | Arpejo ascendente.                         |

A **trilha sonora** é um loop melódico em Lá menor pentatônica, com uma linha de baixo seguindo a progressão Am-F-C-G, em volume reduzido para ficar em segundo plano. A escolha por áudio procedural garante coerência com a estética "digital/luminosa" do jogo e elimina dependências de assets. Um pequeno *fade* nas pontas do loop evita estalos na repetição. A tecla **M** ativa/desativa o som.

## 8. Uso de assets, sprites e animações na Unity

A arte de Lumen é **original e gerada por código** (procedural), com paleta **cyberpunk neon** (ciano, magenta e amarelo sobre fundo escuro). A `SpriteFactory` desenha cada sprite pixel a pixel em uma `Texture2D` (16×16, e 48×48 no chefe) e a converte em `Sprite` com filtro *Point*, resultando em uma estética *pixel art* limpa. São gerados: o personagem (quadros de parado, andar, pular, escalar e atirar), bots e *drones* voadores, o **chefe**, data-shards, armas, projéteis, lajes com faixa neon, escadas, espinhos de energia, o portal de saída e um **fundo de cidade** com silhuetas de prédios, janelas acesas e céu que muda de cor por fase (com *flicker* neon controlado pelo `SkyController`).

### Animação

As animações usam **troca de quadros** (sprite-sheet animation) - a mesma técnica por trás da ferramenta de Animação da Unity, porém dirigida por um componente próprio (`SpriteAnimator`) que troca o `Sprite` do `SpriteRenderer` em uma taxa definida. O `PlayerController` seleciona a animação conforme o estado físico (parado, andando, correndo, pulando, caindo, escalando), e a velocidade da animação de caminhada aumenta ao correr, reforçando a leitura de movimento.

**Créditos de assets:** toda a arte e todo o áudio foram criados proceduralmente pelo próprio código do projeto (arte e som originais). A fonte de texto do HUD é a fonte interna da Unity (`LegacyRuntime.ttf`). Caso se optasse por assets externos gratuitos, o pacote *Kenney Platformer Pack* (kenney.nl, licença CC0) seria a referência recomendada e exigiria atribuição de crédito.

## 9. Conexão entre teoria e prática

A tabela a seguir resume como cada conceito teórico se materializou no código do jogo:

| Conceito teórico                | Aplicação prática em Lumen                                                                |
|---------------------------------|-------------------------------------------------------------------------------------------|
| Game feel / controle responsivo | Coyote time, jump buffer e gravidade variável no <code>PlayerController</code> .          |
| Física de plataforma            | <code>Rigidbody2D</code> + detecção de chão por <code>OverlapBox</code> .                 |
| Progressão de dificuldade       | Quatro mapas ASCII com elementos recombinaados em densidade crescente.                    |
| Interatividade e feedback       | Resposta visual (partículas), sonora (SFX) e sistêmica (vidas/respawn) imediata.          |
| HUD                             | Canvas com vidas, cristais e pontuação atualizados por eventos.                           |
| Game loop                       | Uso correto de <code>Update</code> / <code>FixedUpdate</code> / <code>LateUpdate</code> . |
| Arquitetura de software         | Singletons, responsabilidade única e abordagem data-driven.                               |
| Áudio                           | Síntese procedural de trilha e efeitos coerentes com a estética.                          |

## Conclusão

Fazendo o Lumen eu percebi, na prática, que um jogo de plataforma 2D legal de jogar depende menos de recurso sofisticado e mais de caprichar no básico: controle responsivo, leitura visual clara, dificuldade bem dosada e feedback na hora. Pra mim, o que mais valeu foi montar o jogo de forma dirigida por código e gerar a arte e o áudio por conta própria - no começo deu trabalho (a parte da física do pulo e da colisão me deu uns nós), mas no fim consegui entregar uma experiência completa, que roda em qualquer PC e que eu fico tranquilo de mostrar como portfólio.

## Referências

Unity Technologies. *Unity Learn - Get Started*. Disponível em: [unity.com/learn](https://unity.com/learn).  
Maddy Makes Games. *Celeste*, 2018 (referência de game feel e mecânica de escalada).  
Askiisoft. *Katana ZERO*, 2019 (estética neon/synthwave e ação 2D).  
CD Projekt RED. *Cyberpunk 2077*, 2020 (referência estética do tema cyberpunk).  
Nintendo. *Super Mario Bros.*, 1985 (fundamentos do gênero plataforma).  
Kenney. *Game Assets (CC0)*. Disponível em: [kenney.nl/assets](https://kenney.nl/assets).